Dockerfile Notes

A **Dockerfile** is a text file that contains instructions to build a Docker image.

Common instructions:

```
FROM
WORKDIR
COPY
RUN
EXPOSE
CMD
```

---

1) Single-Stage Dockerfile

A **single-stage Dockerfile** uses **one `FROM` instruction**.

It builds and runs the application in the **same image**.

This is simple and best for **small practice projects**.

Example 1: Python Flask App

Project structure:

```
mypythonapp/
│
├── app.py
├── requirements.txt
└── Dockerfile
```

`app.py`

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello Docker"

app.run(host='0.0.0.0', port=5000)
```

---

requirements.txt

```
flask
```

Single-stage Dockerfile

```
FROM python:3.9

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

EXPOSE 5000

CMD ["python", "app.py"]
```

How it works

- `FROM python:3.9` → base image
- `WORKDIR /app` → set working folder
- `COPY . .` → copy files
- `RUN pip install` → install dependencies
- `EXPOSE 5000` → application port
- `CMD` → run app

Build and Run

```
docker build -t mypythonapp .
docker run -d -p 5000:5000 mypythonapp
```

Problem with Single Stage

Image size becomes **large** because it includes:

- source code
- build tools
- dependencies
- temporary files

Example:

- Python app → 400MB+
- Java app → 700MB+

---

2) Multi-Stage Dockerfile

A **multi-stage Dockerfile** uses **multiple `FROM` instructions**.

It separates:

- **build stage**
- **runtime stage**

This makes the image **smaller and production-ready**.

Example 2: Java App

Project structure:

```
myjavaapp/
│
├── App.java
└── Dockerfile
```

`App.java`

```
public class App {
    public static void main(String[] args) {
        System.out.println("Hello Docker Multi Stage");
    }
}
```

Multi-stage Dockerfile

```
# Stage 1 - Build
FROM openjdk:17 AS builder

WORKDIR /app

COPY App.java .

RUN javac App.java
```

```
# Stage 2 - Run
FROM openjdk:17-jdk-slim

WORKDIR /app

COPY --from=builder /app/App.class .

CMD ["java", "App"]
```

How it works

Stage 1 → Builder

```
FROM openjdk:17 AS builder
```

This stage compiles Java code.

```
RUN javac App.java
```

Creates:

```
App.class
```

Stage 2 → Runtime

```
FROM openjdk:17-jdk-slim
```

This image is lightweight.

Only the compiled file is copied:

```
COPY --from=builder /app/App.class .
```

So final image contains only:

- `App.class`
- Java runtime

No source code, no compiler tools.

Build and Run

```
docker build -t myjavaapp .
docker run myjavaapp
```

Output:

```
Hello Docker Multi Stage
```

Difference Between Single and Multi Stage

Feature	Single Stage	Multi Stage
`FROM`	One	Multiple
Image Size	Large	Small
Build + Run	Same image	Separate
Best For	Practice	Production
Speed	Normal	Better
Security	Less	More

Interview Answer (Important)

What is single-stage Dockerfile?

A Dockerfile with only one `FROM` instruction where the build and runtime happen in the same image.

What is multi-stage Dockerfile?

A Dockerfile with multiple `FROM` instructions where one stage is used to build the application and another stage is used to run it, reducing image size.

Real-Time Example

Single stage

Like carrying:

- books
- laptop
- charger
- bag

- food

everything together.

Multi stage

Like preparing food in kitchen and only serving the final dish 🍽

Kitchen tools stay behind.

Only final output is delivered.

This is exactly how multi-stage works.